Partial Identification and Estimation in a Class of Potential Outcomes Models: A Convex-Fractional Approach

Code guide updated November 13, 2026

1 Overview

The current code computes bounds on the always-takers' treatment effect $\beta = \mathbb{E}[Y(1) - Y(0) \mid S(1) = S(0) = 1]$ without monotonicity assumption, using the estimation procedure outlined in Section 5 and inference in Section 6. Applications include JobCorps (Lee [2009], Section 8), and Thinking Healthy Program (THP, Baranov et al. [2020]) and the Primary School Deworming Project (Miguel and Kremer [2004], Section 9). Some implementation choices are recorded in the file `ASSUMPTIONS.md`.

We separate the scripts into three categories:

1. Data preparation (`data_prep.py`, `data_prep_baranov.py`, `data_prep_deworming.py`): each corresponding file loads the data set, and maps it to treatment D, selection S, outcome Y, covariates X, weights and clusters.

2. Estimation (`estimator.py`, `lp_bounds.py`): the first file includes the first stage estimation, the cross-fitted localized one-step estimator, and inference; the second file computes the classical Lee bounds, exact marginal (no-covariate) bounds, and the discrete LP heuristic of Section 2.3.

3. Runner scripts (`run_*.py`): each script produces a set of tables, prints them out, and stores in the CSV format.

Script	Output	Output files	Paper reference
`run_section8.py`	JobCorps share bounds with bands; bounds by identification regime; complier rescaling; Masten–Poirier frontier	`table1_validation.csv`, `table5_bounds.csv`, `table7_compliers.csv`, `mp_frontier.csv`	Tables 1, 5, 7 (Sections 3.1, 8)
`run_table2.py`	JobCorps discrete-heuristic LP bounds by number of wage bins	`table2_lp_granularity.csv`	Table 3 (Sections 2.3, 3.2)
`run_baranov.py`	THP: ITT, Lee, exact marginal and covariate bounds for every outcome	`baranov_bounds.csv`	Tables 8A, 9 (Section 9)
`run_deworming.py`	Deworming: ITT, Lee, exact marginal and covariate bounds; share bounds by specification	`deworming_bounds.csv`, `deworming_table1.csv`	Tables 8C, 11 (Section 9)
`run_app_tables.py`	THP analogues of Tables 1 and 3	`app_table1_analogues.csv`, `app_table3_analogues.csv`	not in the paper

How to run the code. Each runner script starts from the folder that contains the scripts, e.g. `python run_section8.py`. All runner scripts create and write to `../results/`, a path relative to the *working* directory. Note that the data paths are computer-dependent, stored in the data-preparation modules (`DATA_PATH`, `CSV`), and therefore must be edited once the files are run on another computer. Approximate running times on a M1 MacBook Pro: `run_section8.py` about 30 minutes (of which Table 1 bands with Lee's 28 covariates take about 23), `run_table2.py` under one minute, `run_deworming.py` about 40 seconds.

2 Data source

JobCorps (`dataLee2009.csv`, path `DATA_PATH` in `data_prep.py`). This is the Lee [2009] sample of the National Job Corps Study: 9,145 applicants (5,546 program group, 3,599 control group), 1,219 columns. The variables are:

- `TREATMNT.y`: random assignment to the program group (D); `DSGN_WGT.x`: study design weights (range 4.5–20, mean 10.6); `ENROLL`: enrollment, used for the compliance rate of Table 7;
- `EARNHw`, `HWHw`: weekly earnings and weekly hours in week $w \in \{45, 90, 100, 110, 150, 208\}$; selection $S_w = 1\{\texttt{EARNH}_w > 0,\ \texttt{HWH}_w > 0\}$ and outcome $Y_w = \log(\texttt{EARNH}_w/\texttt{HWH}_w)$ (log hourly wage);
- baseline covariates: the 28 variables in `LEE28`, the eight "reasons for joining" items in `REASONS_ITEMS`, baseline annual earnings `EARN_YR`, household income `HH_INC` and its bracket dummies `HH_INC1`–`HH_INC5`, and `HISP`, `NCHLD`, `MARRIED` for the discrete Table 1 columns.

Thinking Healthy Program (`THP_clean.csv`, path `CSV` in `data_prep_baranov.py`). This is the cleaned data of the Baranov et al. [2020] replication package (`maternalDepression-1.2/dataClean`). The code keeps the 903 cluster-randomized, prenatally depressed mothers (463 treated, 440 control) and drops the 300 non-randomized, non-depressed comparison mothers. The variables are:

- `THP_sample`: randomized-sample indicator (rows with `THP_sample==1` are kept); `treat`: offer of the Thinking Healthy Program (D); `uc`: union council, the unit of randomization (40 clusters, 20 treated and 20 control); no weights ($w = 1$);
- `attrit2`: attrition at the 7-year follow-up; selection $S_7 = 1\{\texttt{attrit2} = 0\}$ (585 of 903 re-interviewed); for the 6-month and 1-year waves, selection is non-missingness of `depindex_6m` and `depindex_1y`;
- outcomes: the 7-year indices in `OUTCOMES_7Y` (`motherfinancial`, `motherfinancial_7y`, `parentmoney`, `parenttime`, `parentstyle`, `healthindex`, `cognindex`, `emoindex`, `depindex_7y`) and the depression indices `depindex_6m`, `depindex_1y` in `OUTCOMES_EARLY`, standardized to control mean 0 and standard deviation 1 and set to missing when $S = 0$;
- baseline covariates: the 15 variables in `X_CONTROLS` (the paper's attrition-IPW control list without `doi0`), plus `age_baseline_sq` derived from `age_baseline`; the six variables in `X_SMALL` for the covariate column; `grandmother_baseline`, `first_child` and `wealth_baseline` for the discrete cells of the Table 1 and Table 3 analogues.

Deworming (`deworming_clean.csv`, path `CSV` in `data_prep_deworming.py`). This is the pupil-level file built from the Miguel and Kremer [2004] replication data: 29,790 pupils in 75 schools (groups 1/2/3: 10,039/10,239/9,512 pupils, 25 schools each), 23 columns. The code keeps the 1998 test frame, the 9,905 pupils with non-missing `sex`. The variables are:

- `treat98`: equal to 1 for phase-in groups 1 and 2 and 0 for group 3 (D); `wgrp`: phase-in group (in 1998 only group 1 was treated); `schid`: school, the unit of randomization (75 clusters); `pupid`: pupil identifier; no weights ($w = 1$);
- `tested98`: present on the 1998 ICS test day, the selection S; `testscore98`: combined English/math/science ICS score standardized to group 3, the outcome Y (observed when $S = 1$); `ics98`: the same score before rescaling;
- baseline covariates: pupil variables `sex`, `age`, `grade`, `ses_index` in `X_PUPIL` (`age` and `ses_index` missing for 34 and 11 pupils, median-imputed) and school variables `z_inf98` (baseline infection) and `pup_pop` (school enrollment) in `X_SCHOOL`; `sex` and `grade` also define the discrete cells of the Table 1 analogue.

3 Overview of scripts

The table lists every function and class method, grouped by file, in the order data preparation, estimation, and runner scripts. Functions defined inside another function are marked "nested".

Function	Input	Output	Description
`data_prep.py`			
`load_data`	none; reads the file at `DATA_PATH`	data frame with 9,145 rows	reads the JobCorps CSV file and asserts the Lee [2009] sample size $N = 9{,}145$
`outcome_selection`	data frame `df`; follow-up week `week`	tuple `(S,Y)` of length-N arrays; `Y` is `NaN` where $S = 0$	builds selection indicator $S_w = 1\{\texttt{EARNH}_w > 0,\ \texttt{HWH}_w > 0\}$ (employed with recorded earnings and hours in week w) and outcome $Y_w = \log(\texttt{EARNH}_w/\texttt{HWH}_w)$, the log hourly wage, observed only when $S_w = 1$; non-numeric entries are treated as missing, so $S = 0$
`treatment`	data frame `df`	integer array D	random-assignment indicator `TREATMNT.y` (1 = program group)
`weights`	data frame `df`	float array	study design weights `DSGN_WGT.x`, used in every design-weighted computation of Section 8
`earnings_cells`	data frame `df`	integer array of cell labels	bins baseline annual earnings `EARN_YR` into five quantile cells (`pd.qcut`, duplicate edges dropped); the point mass at zero leaves four distinct cells; missing `EARN_YR` forms cell -1; "Earnings" column of the LP table
`income_cells`	data frame `df`	integer array of cell labels	household income `HH_INC` on its native brackets, missing coded -1 (five recorded brackets plus missing); "Income" column of the LP table
`design_matrix`	data frame `df`; list of column names `columns`; `add_missing_dummies` (default `True`)	tuple `(X,names)`: $N \times p$ float matrix and the names of the retained columns	builds the covariate matrix: `MOSTIMPR` is expanded into category dummies (first level dropped); every other column is median-imputed and, if it has more than two distinct values, winsorized at its 1st/99th percentiles; a missingness indicator `<col>_miss` is appended for each column with missing values; constant columns are dropped; called with `REASONS_ITEMS`, `LEE28` and the earnings-plus-income set
`data_prep_baranov.py`			
`load_randomized`	none; reads the file at `CSV`	tuple `(r,D,uc,S7)`: data frame, treatment, cluster ids, 7-year response indicator	keeps the randomized mothers (`THP_sample==1`; asserts 903 rows); $D =$ `treat`, clusters = union councils `uc` (40), $S_7 = 1\{\text{attrit2} = 0\}$
`design_matrix`	data frame `r` from `load_randomized`	tuple `(X,names)`: 903×16 matrix and column names	full baseline covariate matrix from `X_CONTROLS` (asserts no missing values), continuous covariates winsorized at 1%/99%, plus `age_baseline_sq` computed from the unwinsorized age; constant columns dropped; used for the "Logit-full" specification in `run_app_tables.py`
`outcome`	data frame `r`; outcome column `col`; selection indicator `S`	float array, `NaN` where $S = 0$	returns the outcome index and blanks it for non-respondents, following the replication package's convention before `leebounds`
`wave_selection`	data frame `r`; outcome column `col`	integer array	selection for the 6-month and 1-year waves, defined as non-missingness of the outcome column
`data_prep_deworming.py`			

Python function	Input	Output	Description
`load_frame`	none; reads the file at `CSV`	tuple `(df,D,schid,S,Y)`	keeps the 1998 test frame (non-missing `sex`, $N = 9{,}905$); $D =$ `treat98`, clusters = schools `schid` (75), $S =$ `tested98`, $Y =$ `testscore98` for tested pupils and `NaN` otherwise
`design`	data frame `df`; list of columns `cols`	tuple `(X,names)`	median-imputes each column (no missingness indicators), winsorizes columns with more than two distinct values at 1%/99%, drops constant columns; called with `X_PUPIL+X_SCHOOL`
`estimator.py`			
`_standardize`	matrix `X`	tuple `(Xs,mu,sd)`	column-wise standardization; zero standard deviations are replaced by 1
`Logit`	penalty parameter `C` (default 1.0)	object	class for the ℓ_1-penalized logistic regression (5.3) used for $p(x)$, $s_0(x)$, $s_1(x)$ and the share bounds of Table 1
`Logit.fit`	covariates `X`; binary outcome `y`; weights `w` (default `None`)	the fitted object	standardizes `X` and fits `LogisticRegression` (`penalty="l1"`, `solver="liblinear"`) with sample weights; if `y` has a single class or the design is constant, stores the weighted mean of `y` instead
`Logit.predict`	covariates `X`	vector of fitted probabilities	applies the stored standardization and returns $\widehat{\mathbb{P}}(y = 1 \mid x)$, or the stored constant
`QuantileCurves`	penalty `alpha` (default 10^{-3})	object	class for the ℓ_1-penalized conditional quantile curves (5.4)
`QuantileCurves.fit`	covariates `X`; outcome `y`; weights `w` (default `None`)	the fitted object	fits one `QuantileRegressor` (HiGHS) on standardized covariates at each of the 51 levels in `U_GRID`; stores the outcome range of the fitting sample
`QuantileCurves.predict`	covariates `X`	$n \times 51$ matrix `Q`, $Q[i,j] = \widehat{F}^{-1}(u_j \mid x_i)$	evaluates all quantile fits, rearranges each row into increasing order (Section 5.2) and clips to the observed outcome range
`quantile_at`	fitted curves `Q` ($n \times 51$); levels `u` (scalar or n-vector)	n-vector $\widehat{F}^{-1}(u_i \mid x_i)$	linear interpolation along `U_GRID`; flat extrapolation below 0.005 and above 0.995
`lower_partial_int`	fitted curves `Q`; upper limits `a` (n-vector in $[0,1]$)	n-vector $\int_0^{a_i} \widehat{F}^{-1}(u \mid x_i)\,du$	trapezoid rule on the interpolated curve with constant extrapolation at both ends; gives the trim integrals $\psi_d(t,x) = s_d \int_0^{t/s_d} F_d^{-1}(u \mid x)\,du$
`CellProgram`	quantile curves `Q1`, `Q0`; selection probabilities `s1`, `s0`; box endpoints `piL`, `piU` at n points; grid size `n_t` (default 101)	object	precomputes, at each point, a grid of `n_t` shares $t \in [\pi_L(x), \pi_U(x)]$ and the quantile treatment effect $\mathrm{QTE}(t,x) = F_1^{-1}(t/s_1) - F_0^{-1}(1 - t/s_0)$ of (5.22), made monotone by a cumulative maximum
`CellProgram.pi_star`	multiplier `lam`	n-vector $\pi^*(x,\lambda)$	solves $\mathrm{QTE}(t,x) = \lambda$ by linear interpolation on the grid; values outside the grid's QTE range go to the floor or ceiling of the box
`CellProgram.trim_value`	shares `t` (n-vector)	n-vector $n(t,x)$	per-cell numerator (5.21): lower trim of the treated outcome at mass t minus upper trim of the control outcome at mass t
`CellProgram.thresholds`	shares `t`	tuple `(tau1,tau0,a1,a0)`	trim thresholds $\tau_1 = F_1^{-1}(t/s_1)$, $\tau_0 = F_0^{-1}(1 - t/s_0)$ and the fractions $a_1 = t/s_1$ (kept) and $a_0 = 1 - t/s_0$ (discarded), used to detect when a trim keeps a whole arm

Python function	Input	Output	Description
`dinkelbach_root`	`CellProgram` `cp`; weights `wts`; full sample size `N_total`; tolerance `tol` (default $N^{-1/4}/\log N$)	tuple `(lam_init, status)`; status `"ok"`, `"degenerate"` (returns 0) or `"undefined"` (returns `NaN`)	Algorithm 5.1, bisection variant: root of $G(\lambda) = \overline{n(\pi^*(\lambda))} - \lambda\overline{\pi^*(\lambda)}$ (weighted mean) on the bracket $[\min_i(Q_1(0.005) - Q_0(0.995)),\ \max_i(Q_1(0.995) - Q_0(0.005))]$; stops when $\lvert G\rvert \le$ `tol` $\cdot$ mean π^* (at most 200 steps); `"undefined"` when the weighted mean of π_L is not positive, `"degenerate"` when G has no sign change
`G`	multiplier `lam`	tuple (value of G, π^*)	nested in `dinkelbach_root`: evaluates the plug-in Dinkelbach moment (5.5)
`buffer_delta`	sample size `N`	scalar	buffer $\delta_N = N^{-1/4}/\log N$ of (5.19)
`classify`	`pi_star`; `piL`; `piU`; buffer `dN`	integer array: 0 interior, 1 floor, 2 ceiling	buffer rule (5.20); a point within δ_N of both faces is assigned to the nearer one, the floor on ties
`signals`	`Y`, `S`, `D`, `w`; fitted `p`, `s0`, `s1`; box `piL`, `piU`; frozen `pi_star`, `tau1`, `tau0`; hinge fits `g1`, `g0`; `regime`; optional `floor_face`, `a1`, `a0`	dict of per-observation arrays `eta1`, `eta0`, `psi_t`, `psi_t0`, `psi_pi`, `A`	orthogonal signals of Section 5.3 for the difference target: AIPW numerators η_1 (treated lower trim, (5.8)) and η_0 (control upper trim); type signals ψ_{t1}, ψ_{t0} (5.6), with residuals centered at the fitted kept mass; floor and ceiling signals ψ_{LB} (5.13) and ψ_{UB} (5.14); active-facet selector ψ_π (5.15); numerator summand $A = \eta_1 - \eta_0 + \tau_1(\psi_\pi - \psi_{t1}) - \tau_0(\psi_\pi - \psi_{t0})$ (`ASSUMPTIONS.md` item 13); with `floor_face` the floor face is ψ_{LB} (code 0), the constant share (1) or ψ_{UB} (2)
`first_stage_cache`	`X`, `Y`, `S`, `D`, `w`; fold labels `folds` (three folds); `qr_alpha` (10^{-3}); `logit_C` (1.0); clipping `clip_s` (0.01), `clip_p` (0.05)	list of three dicts, one per rotation: fold masks `J1`, `J2`, `J3`; predictions `p`, `s0`, `s1`, `Q0`, `Q1` for all observations	cross-fitting of Definition 5.1: in rotation r the propensity p and selection probabilities s_d are fitted on J_2 (`Logit`), and the quantile curves on the selected units of each arm in J_1 (`QuantileCurves`); s_d clipped to $[0.01, 0.99]$, p to $[0.05, 0.95]$; computed once and reused for both endpoints and every Masten–Poirier floor
`_box`	raw endpoints `piL_raw`, `piU_raw`; Masten–Poirier floor `floor_bar` (default `None`); `collapse_ceiling` (default `False`)	tuple `(piL, piU)`	applies the pointwise floor $\pi_L^{new} = \min\{\max(\pi_L, \bar\pi), \pi_U\}$ of Section 8, point 3, or collapses the box to the ceiling (conditional monotonicity, Appendix A)
`one_sided_bound`	`X`, `Y`, `S`, `D`, `w`; `cache`; `negate` (default `False`); `lasso_alpha` (10^{-3}); `floor_bar`; `collapse_ceiling`	dict: `estimate`, `status`, `lam_inits`, per-observation `A`, `psi_pi`, `psi_t`, `tau_diff`, `regime`, per-rotation `rot_estimates`	localized one-step (Algorithm 5.2, Section 5.4.3) for the lower endpoint, or for the lower endpoint of $-Y$ when `negate=True`; in each rotation: λ_{init} on J_1 (`dinkelbach_root`); π^*, τ_1, τ_0 fixed at λ_{init}; Lasso hinge regressions $g_1 = \mathbb{E}[Y1\{Y \le \tau_1\} \mid D{=}1, S{=}1, X]$ and $g_0 = \mathbb{E}[Y1\{Y \ge \tau_0\} \mid D{=}0, S{=}1, X]$ on J_2; signals on J_3; the estimate is the pooled ratio $\sum wA / \sum w\psi_\pi$ over the three J_3 folds; returns status `"undefined"`, `"degenerate"` or `"degenerate-denominator"` (a rotation with non-positive denominator) instead of an estimate
`frozen`	index mask `idx`	tuple of s_0, s_1, box, π^*, τ_1, τ_0, a_1, a_0 at the selected points	nested in `one_sided_bound`: evaluates the per-cell program at λ_{init} on a fold

Python function	Input	Output	Description
`bound_interval`	`X`, `Y`, `S`, `D`, `w`; `cache`; `alpha` (0.05); `n_boot` (400); `seed` (11); `cluster` (default `None`); further arguments passed to `one_sided_bound`	dict: `lower`, `upper`, `se_lower`, `se_upper` (plug-in), `se_boot_lower`, `se_boot_upper`, bootstrap quantiles, `cov_boot_LU`, `IM_lo`, `IM_hi`, `q`, `regime_shares`, `lam_inits`, `piAT`; or status `"degenerate/undefined"`	runs `one_sided_bound` on Y and on $-Y$, negates the second and sorts the endpoints (Algorithm 5.2, step 4); plug-in influence-function variance (Theorem 6.1(iii)) and the multiplier bootstrap of Remark 6.3 (Exp(1) weights, Ψ_i and $\hat{\pi}_{AT}$ held fixed; one weight per cluster and a $G/(G-1)$ correction when `cluster` is given); Imbens–Manski region (6.3) with the bootstrap standard errors and $q = \Phi^{-1}(\sqrt{1-\alpha})$
`_wquantile`	values `y`; weights `w`; level `q`	scalar	weighted quantile by linear interpolation of the cumulative weight distribution
`basic_lee`	`Y`, `S`, `D`, `w`; `n_boot` (400); `alpha` (0.05); `seed` (11)	dict: `lower`, `upper`, `se_lower`, `se_upper`, `IM_lo`, `IM_hi`	Lee bounds without covariates under $S(1) \geq S(0)$ (Table 5, column 1): the treated arm is cut at the weighted quantiles that keep a share s_0/s_1 and compared with the control mean; when $s_1 < s_0$ no trimming is done and both endpoints equal the difference in means; i.i.d. bootstrap standard errors; Imbens–Manski region
`endpoints`	`Y`, `S`, `D`, `w`	tuple `(lo,hi)`	nested in `basic_lee`: point bounds on one (bootstrap) sample
`_exact_root`	selected outcomes and weights `y1`, `w1`, `y0`, `w0`; selection rates `s1`, `s0`; share box `lo_t`, `hi_t`	scalar lower bound	exact single-cell Dinkelbach root: $n(t)$ is piecewise linear with kinks where the cumulative mass of an arm crosses an atom, so $G(\lambda) = \min_t[n(t) - \lambda t]$ is evaluated on those candidate masses, with fractional trimming of the boundary atom; bracket expansion followed by 80 bisection steps; nested helpers `psi` (trim integral) and `G`
`exact_marginal_bounds`	`Y`, `S`, `D`, `w`; `cluster` (default `None`); `n_boot` (400); `alpha` (0.05); `seed` (11)	dict: `status`, `lower`, `upper`, `se_boot_lower`, `se_boot_upper` (also stored as `se_lower`, `se_upper`), `IM_lo`, `IM_hi`, `exact`; or status `"undefined"` when $s_0 + s_1 \leq 1$	sharp no-covariate, no-monotonicity bounds: lower endpoint from `_exact_root`, upper endpoint from `_exact_root` on $-Y$; nonparametric (cluster) bootstrap in which resamples with zero floor are dropped; Imbens–Manski region; used for the "no monotonicity, marginal" columns of THP and deworming; nested helper `point`
`lp_bounds.py`			
`cell_moments`	`Y`, `S`, `D`; cell labels `cells`; number of bins `K`	dict: `edges`, effective `K`, `J`, cell shares `w`, selection rates `s` ($2 \times J$), masses `m` ($2 \times J \times K$), `piL`, `piU`, `cell_ids`	equal-mass outcome bins from the pooled selected sample (duplicate edges removed); cell shares w_j, selection rates $s_{d,j}$, gridded masses $m_{d,j,k} = \Pr(Y \in B_k, S{=}1 \mid D{=}d, C_j)$ and the Fréchet endpoints (2.19); unweighted
`_solve_cc`	moments dict `mom`; `lower` (default `True`)	scalar endpoint, or `NaN` if $\sum_j w_j \pi_{L,j} \leq 0$ or the LP fails	Charnes–Cooper LP (2.17) for the two-arm, common-share version of the fractional program: the lower endpoint minimizes $\sum_j w_j(\sum_k \underline{y}_k r_{1,jk} - \sum_k \overline{y}_k r_{0,jk}) / \sum_j w_j \pi_j$ over $0 \leq r_d \leq m_d$, $\sum_k r_{d,jk} = \pi_j$, $\pi_j \in [\pi_{L,j}, \pi_{U,j}]$; conservative bin endpoints (Proposition 2.2); solved with HiGHS
`lp_wage_effect_bounds`	`Y`, `S`, `D`; `cells`; `K`	tuple `(lower,upper,mom)`	computes `cell_moments` and solves both LPs; used by `run_table2.py` and `run_app_tables.py`
`run_section8.py`			

Python function	Input	Output	Description
`fmt`	result dict `res`	string	formats a result as the interval [`lower`, `upper`] followed by the Imbens–Manski region (`IM_lo`, `IM_hi`); returns the status if the run failed
`main`	none	none; writes `table1_validation.csv`, `table5_bounds.csv`, `mp_frontier.csv`, `table7_compliers.csv` to `../results/`	JobCorps application: loads the data and design matrices (Reasons, Lee's 28, earnings plus income); three folds by seeded permutation (seed 42). Table 1: $\mathbb{E}[\pi_L]$, $\mathbb{E}[\pi_U]$ for the None, Hispanic, Kidcount, Married, EarnInc, Lee28 and Reasons specifications at weeks 45, 90, 100, with one-sided 95% lower bands (5th percentile of 400 bootstrap resamples, seed 2027). Prints the weighted compliance rate. Table 5 at weeks 45, 90, 110, 150, 208: (1) `basic_lee`; (2) `bound_interval` with Lee's 28 and `collapse_ceiling=True`; (3) `bound_interval` with Reasons; (4) Masten–Poirier floor at the midpoint of $[\mathbb{E}\pi_L, \mathbb{E}\pi_U]$ from full-sample logits, plus a 7-point frontier. Table 7 divides Table 5 by 0.725; nested helper `rec` formats one Table 5 row
`cell_epi`	`S_`, `D_`, `w_`, `cells_`	tuple $(\mathbb{E}\pi_L, \mathbb{E}\pi_U)$	nested in `main`: design-weighted cell-averaged Fréchet endpoints (2.19); cells with an empty arm contribute zero
`logit_epi`	`S_`, `D_`, `w_`, `X_`	tuple $(\mathbb{E}\pi_L, \mathbb{E}\pi_U)$	nested in `main`: weighted logit fits of $s_0(x)$ and $s_1(x)$ on the full sample, then weighted means of $(s_0 + s_1 - 1)_+$ and $\min(s_0, s_1)$
`cat_cells`	column name `col`	integer cell labels	nested in `main`: a categorical column as cells, missing coded -1
`run_table2.py`			
`main`	none	none; writes `../results/table2_lp_granularity.csv`	discrete-heuristic LP bounds (`lp_wage_effect_bounds`) for $K \in \{5, 10, 15\}$ bins, weeks 45, 90, 100 and the Earnings and Income cells, unweighted; prints the bounds, their width and $\mathbb{E}[\pi_L]$; corresponds to the paper's Table 3
`run_baranov.py`			
`cluster_folds`	cluster ids `uc`; treatment `D`; `n_folds` (3); `seed` (42)	integer fold label per observation	assigns whole clusters to folds, stratified by arm (a separate permutation of treated and control clusters)
`_frac_tail_mean`	values `y`; weights `w`; share `share`; `top` (bool)	scalar	weighted mean of the top (`top=True`) or bottom share of the distribution, with fractional weight on the boundary observation
`lee_bidirectional`	`Y`, `S`, `D`, `w`; cluster ids `uc`; `n_boot` (400); `seed` (11)	dict: `lower`, `upper`, `se_lower`, `se_upper`, `IM_lo`, `IM_hi`, `status`	Lee bounds that trim the arm with the higher selection rate (direction chosen by the data and re-chosen in every bootstrap draw), exact fractional trimming, cluster bootstrap standard errors, Imbens–Manski region with $q = \Phi^{-1}(\sqrt{0.95})$; nested helper `point` computes the bounds on one sample
`itt_respondents`	`Y`, `S`, `D`, `w`; cluster ids `uc`	tuple `(beta,se)`	difference in weighted means among respondents and its cluster-robust standard error with the CR1 small-sample factor; the standard error uses unit weights
`fmt`	result dict `res`	string	as in `run_section8.py`

Python function	Input	Output	Description
`main`	none	none; writes `../results/baranov_bounds.csv`	THP application: stratified cluster folds; for the nine 7-year outcomes (selection S_7) and the 6-month and 1-year depression indices (selection = outcome observed): (1) `itt_respondents`; (2) `lee_bidirectional`; (3) `exact_marginal_bounds`; (4) `bound_interval` with `X_SMALL`, `qr_alpha=1e-2`, clustered on union councils; prints selection rates and regime shares; nested helper `rec` formats one row
`run_deworming.py`			
`cluster_folds`	school ids `schid`; `D`; `n_folds` (3); `seed` (42)	integer fold label per observation	same as in `run_baranov.py`
`main`	none	none; writes `../results/deworming_bounds.csv` and `deworming_table1.csv`	deworming application on the test frame: (1) ITT on tested pupils; (2) `lee_bidirectional`; (3) `exact_marginal_bounds`; (4) `bound_interval` with pupil and school covariates, `qr_alpha=1e-2`, clustered on schools; then $\mathbb{E}[\pi_L]$, $\mathbb{E}[\pi_U]$ for the None, Sex, Grade and logit specifications; imports `lee_bidirectional`, `itt_respondents` and `fmt` from `run_baranov.py`
`cell_epi`	cell labels `cells`	tuple $(\mathbb{E}\pi_L, \mathbb{E}\pi_U)$	nested in `main`: unweighted cell-averaged endpoints; cells with fewer than 5 observations in an arm are skipped
`logit_epi`	covariates `X`	tuple $(\mathbb{E}\pi_L, \mathbb{E}\pi_U)$	nested in `main`: unweighted full-sample logit version
`run_app_tables.py`			
`share_box`	`S`, `D`; cell labels `cells`	tuple $(\mathbb{E}\pi_L, \mathbb{E}\pi_U)$	unweighted cell-averaged Fréchet endpoints; cells with an empty arm skipped
`logit_box`	`S`, `D`; covariates `X`	tuple $(\mathbb{E}\pi_L, \mathbb{E}\pi_U)$	unweighted full-sample logit fits of $s_0(x)$, $s_1(x)$ and the means of $(s_0 + s_1 - 1)_+$ and $\min(s_0, s_1)$
`band`	function `fn` (`share_box` or `logit_box`); `S`, `D`; its argument `arg`; `cluster` (default `None`)	scalar	one-sided 95% lower band for $\mathbb{E}[\pi_L]$: 5th percentile over `N_BOOT` (cluster) bootstrap resamples drawn with the module generator `RNG`; failed resamples skipped
`main`	none	tuple `(t1,t3)` of data frames; writes `app_table1_analogues.csv` and `app_table3_analogues.csv` to `results/` next to the script	THP Table 1 analogue at the 7-year, 6-month and 1-year waves for the None, Grandmother, FirstChild, Logit-targeted (`X_SMALL`) and Logit-full (`design_matrix`) specifications, with bands clustered on union councils; Table 3 analogue: LP bounds with wealth-quintile cells at $K = 5, 10, 15$ for `depindex_6m`, `depindex_1y` and 7-year `parentmoney`; prints both tables

3.1 Module-level constants

Constant	File	Content
`DATA_PATH`	`data_prep.py`	absolute path to `dataLee2009.csv`
`REASONS_ITEMS`	`data_prep.py`	the eight "reasons for joining" items: `R_HOME`, `R_COMM`, `R_TRAIN`, `R_CRGOAL`, `R_GETGED`, `R_NOWORK`, `R_OTHER`, and the most-important-reason code `MOSTIMPR`

Constant	File	Content
LEE28	data_prep.py	28 baseline covariates: demographics (FEMALE, AGE, BLACK, HISP, OTHERRAC), marital status (NEVERMARRIED, MARRIED, TOGETHER, SEPARATED), children (HASCHLD, NCHLD), education (HGC, HGC_MOTH, HGC_FATH), work history and earnings (EVERWORK, YR_WORK, EARN_YR, WKEARNR, WELF_KID), household income brackets HH_INC1–HH_INC5, personal income brackets PERS_INC1–PERS_INC4
CSV	data_prep_baranov.py	absolute path to THP_clean.csv
X_CONTROLS	data_prep_baranov.py	15 baseline covariates (the paper's attrition-IPW control list without doi0): age, employment, empowerment, grandmother present, mother-in-law, wealth, education dummies and years, father's education, number of children, first child, baseline HAM-D and MSPSS
X_SMALL	data_prep_baranov.py	six covariates for the covariate column: mo_emp, employed_mo_baseline, mspss_baseline, wealth_baseline, hamd_baseline, edu_mo_baseline
OUTCOMES_7Y	data_prep_baranov.py	nine 7-year outcomes as (label, column, direction): financial empowerment (pooled and 7-year only), monetary and time investment, parenting style, physical, cognitive and socio-emotional development, depression
OUTCOMES_EARLY	data_prep_baranov.py	depression indices at 6 months and 1 year
CSV	data_prep_deworming.py	absolute path to deworming_clean.csv
X_PUPIL, X_SCHOOL	data_prep_deworming.py	pupil covariates sex, age, grade, ses_index; school covariates z_inf98, pup_pop
U_GRID	estimator.py	51 quantile levels: 0.005, then 0.02, 0.04, . . . , 0.98, then 0.995
WEEKS_T5, SEED	run_section8.py	Table 5 horizons $[45, 90, 110, 150, 208]$; fold seed 42 (inside main: T1_BOOT = 400)
SEED, N_BOOT	run_baranov.py, run_deworming.py	fold seed 42; 400 bootstrap draws
RNG, N_BOOT	run_app_tables.py	random generator with seed 2028; 400 bootstrap draws

References

Victoria Baranov, Sonia Bhalotra, Pietro Biroli, and Joanna Maselko. Maternal depression, women's empowerment, and parental investment: Evidence from a randomized controlled trial. *American Economic Review*, 110 (3):824–59, March 2020.

David S. Lee. Training, wages, and sample selection: Estimating sharp bounds on treatment effects. *Review of Economic Studies*, 76(3):1071–1102, 2009.

Edward Miguel and Michael Kremer. Worms: Identifying impacts on education and health in the presence of treatment externalities. *Econometrica*, 72(1):159–217, 2004.